

Minimum Spanning Tree – Part 2





Review – Lecture 8

Minimum Spanning Tree

Input: Undirected connected graph G = (V,E) and a cost function $c: E \to \mathbb{R}^+$.

Output: A spanning tree T with minimum total cost, $\Sigma_{e \in T}$ c(e).

The blue rule: Find a cut with no blue edge. Color blue the cheapest black edge in the cut.

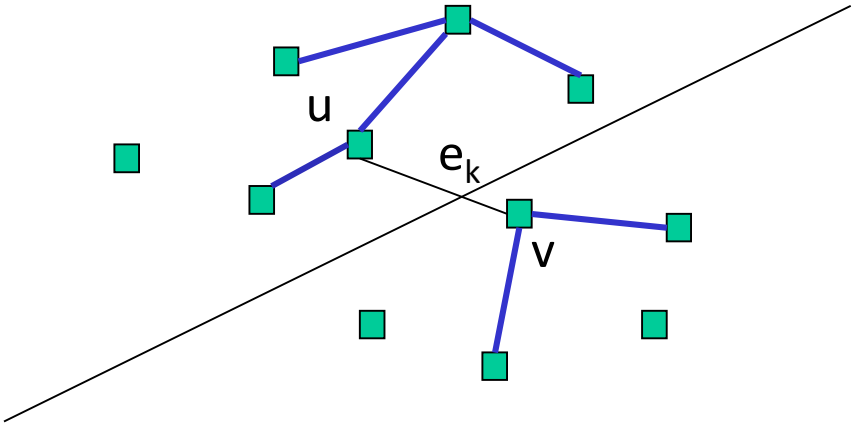
The red rule: Find a cycle with no red edge. Color red the most expensive black edge in the cycle.

General method: Color edges by applying the rules, until n-1 edges are blue.

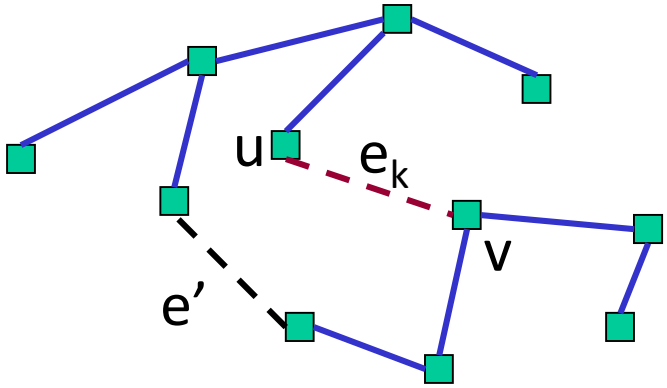
Review – Lecture 8

Theorem: For any $k \geq 0$, after we color k edges, there exists an MST that includes all the blue edges and none of the red edges.

Proof method: Induction. The induction step is based on an exchange argument.



If e_k is colored blue, then it can be added to a tree that does not include it



If e_k is colored red, then it can be removed from a tree that includes it

Review – Lecture 8

Minimum Spanning Tree

Kruskal's Algorithm:

Sort the edges by increasing cost

T=∅

Consider the edges according to the sorted order.

If e does not close a cycle with edges already in T

add it to T

Correctness: This is a possible application of the Blue/Red rules.

Review – Lecture 8

Minimum Spanning Tree

Kruskal's Algorithm:

Implementation: Using Disjoint U/F – Every connected component in T is a set.

A considered edge (u,v) closes a cycle iff u and v belong to the same connected component = to the same U/F set.

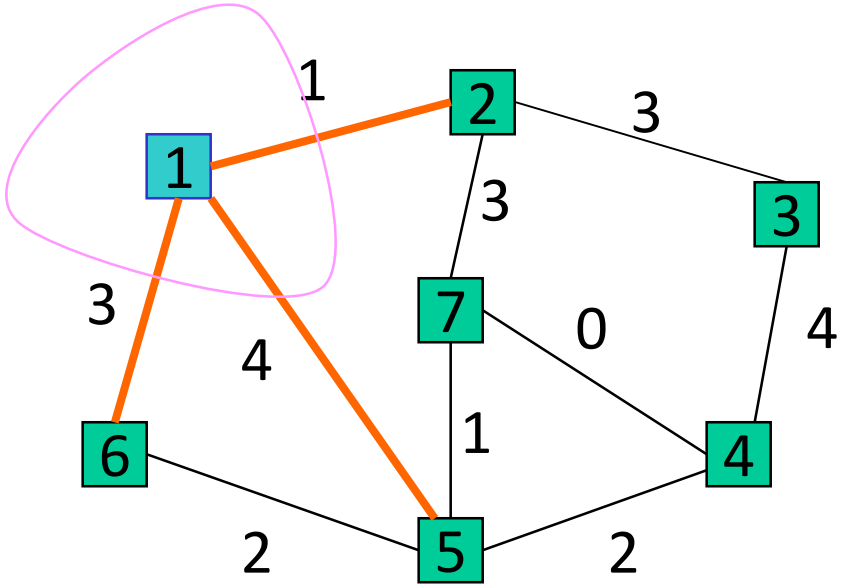
Time complexity: O(m log m).

Prim's Algorithm

- Another algorithm that follows the blue/red rules.
- During the algorithm, the blue edges form a single tree T.
- Every vertex is either 'already in the blue tree' or 'isolated' (none of its adjacent edges is blue).
- Note: In Kruskal's algorithm we may have a forest of many blue trees.
- In Prim's algorithm, initially, the blue tree consists of one vertex and no edges.
- In every iteration we apply the blue rule. The cut is defined by the vertices of the blue tree.

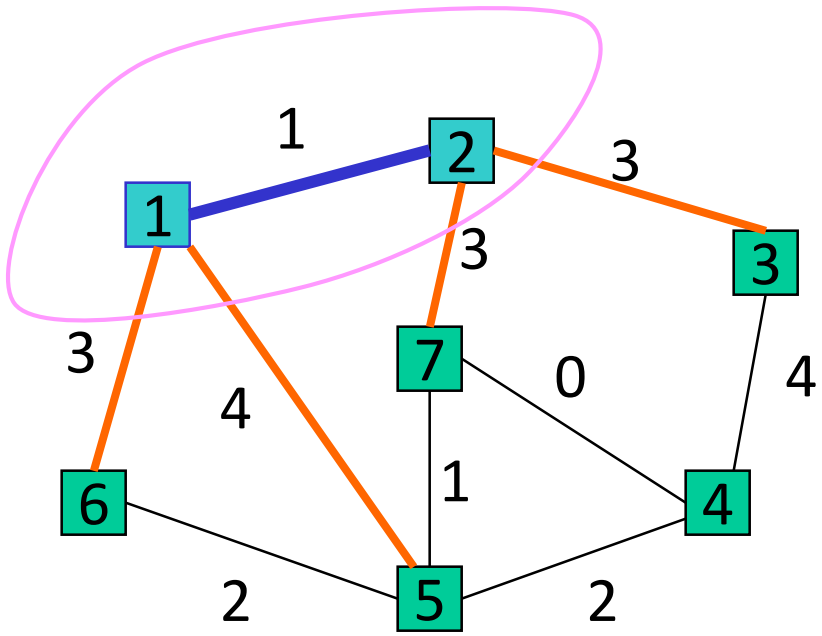
Prim's Algorithm

Example: Assume we begin the run with node 1



1. apply the blue rule on the cut induced by 1

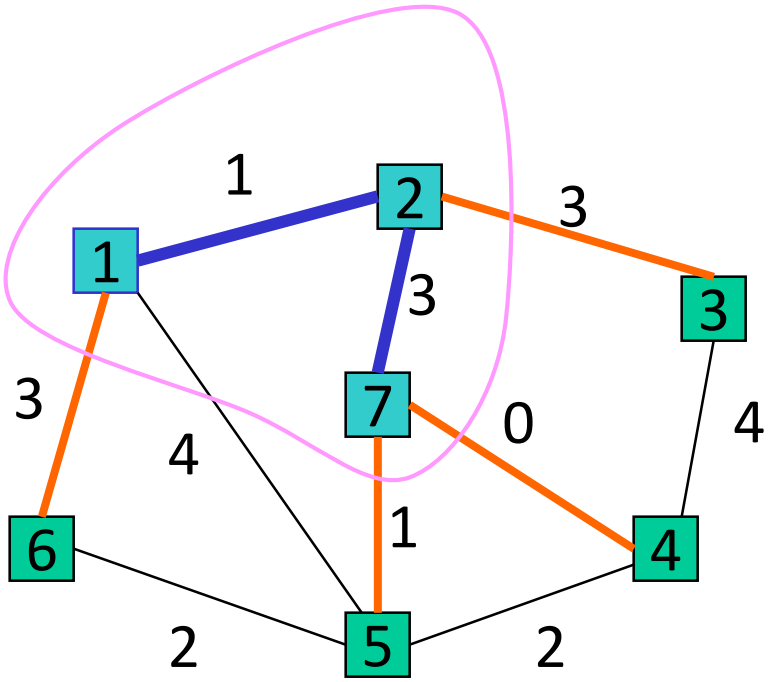
Prim's Algorithm 2



Apply the blue rule on the cut induced by {1, 2}.

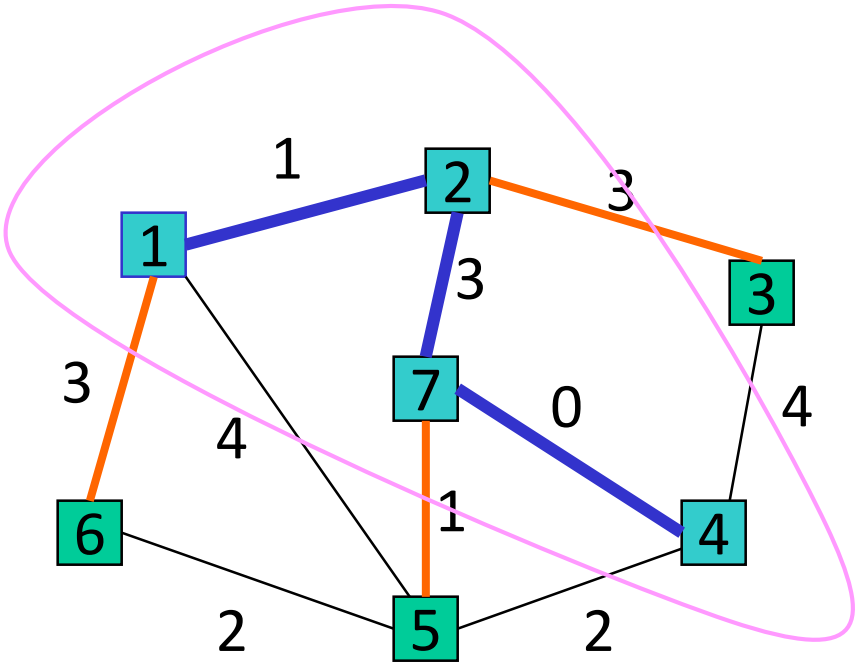
In general: in every iteration we add to T the cheapest edge in the cut induced by the vertices that are already in T.

Prim's Algorithm 3

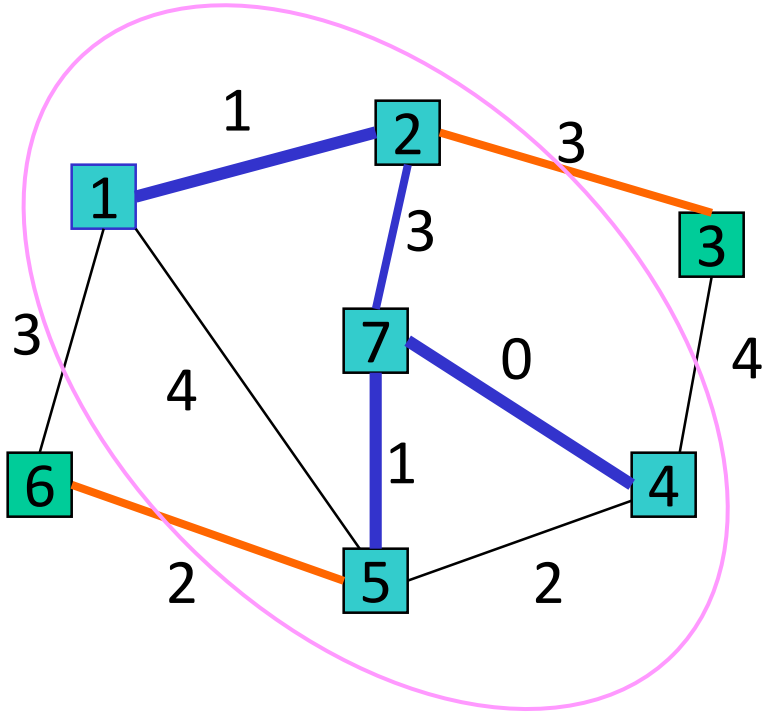


For each vertex not in the tree maintain the cheapest edge to a vertex in the tree (if exists).

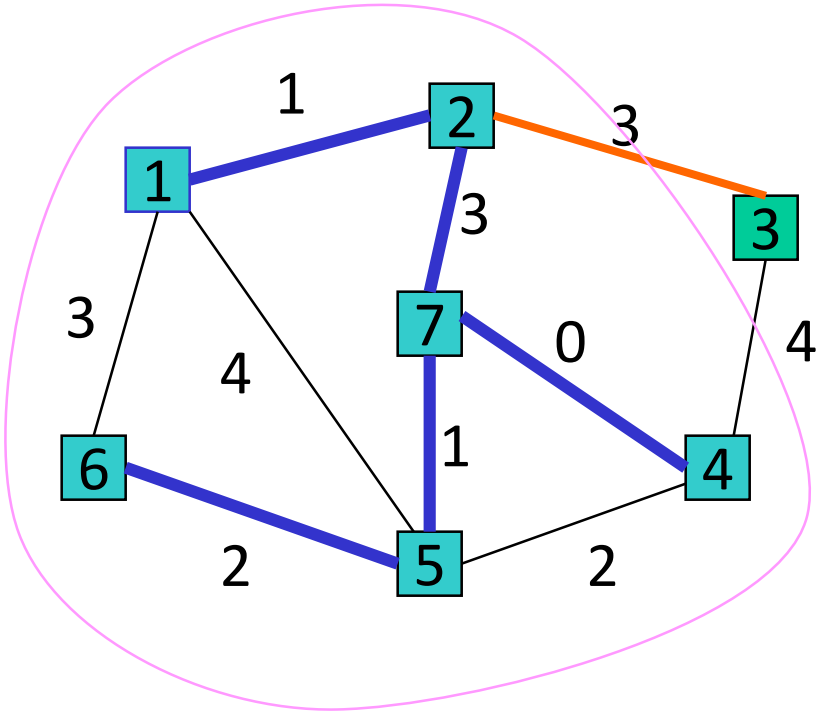
Prim's Algorithm 4



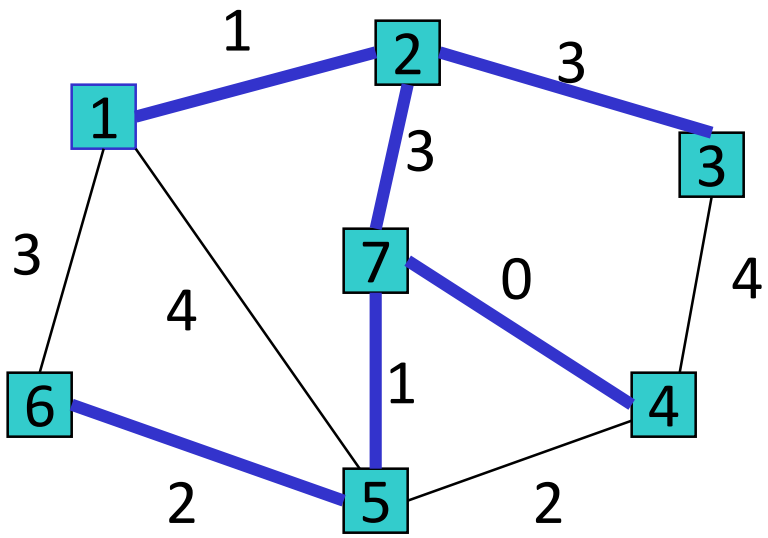
Prim's Algorithm 5



Prim's Algorithm 6



Prim's Algorithm 7



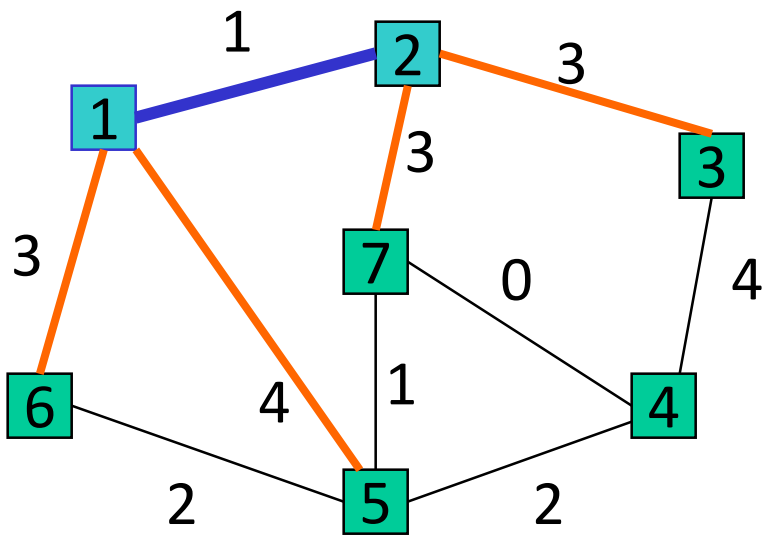
Correctness Proof for Prim

- Repeatedly executes the blue rule (n-1 times).
- In each step we consider the cut defined by the vertices that are already in T.

Data Structures for Prim

In order to implement Prim's algorithms:

1. The graph is given by adjacency Lists – when a node joins T, we need to look at all the edges adjacent to it
2. A best-edge array of size |V| -with two fields per node

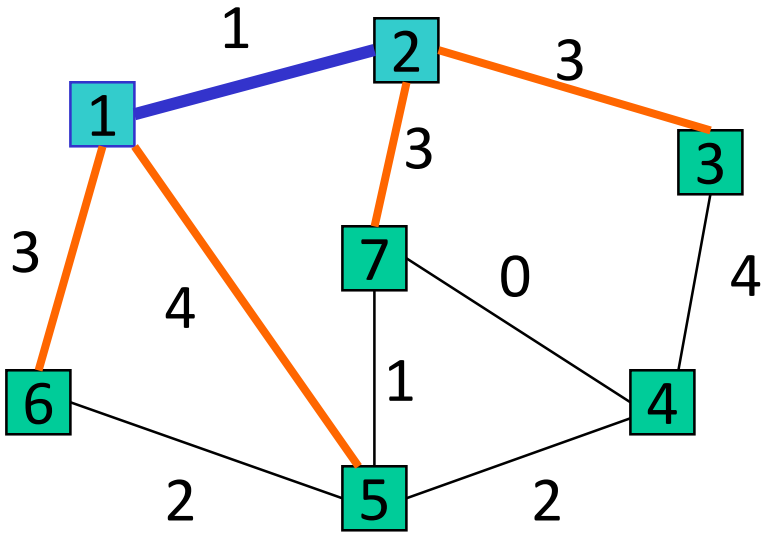


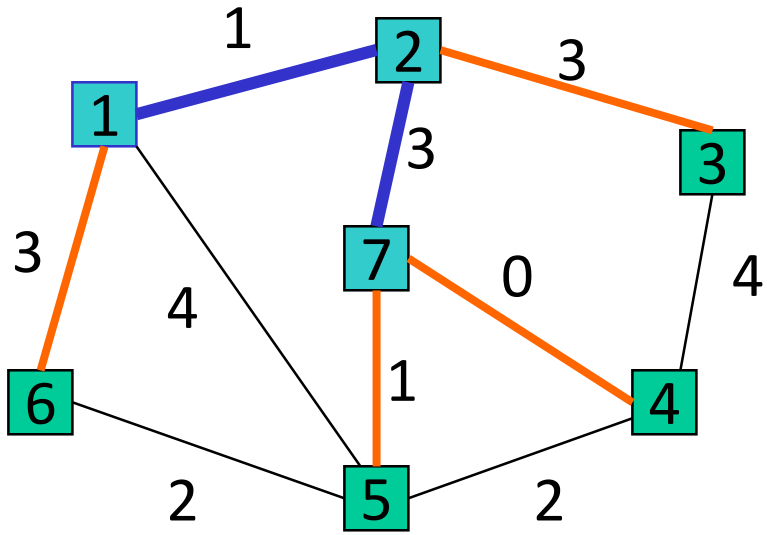
	1	2	3	4	5	6	7
parent			2	-	1	1	2
cost			3	∞	4	3	3

(1,5) is the cheapest edge adjacent to 5 in the cut induced by {1,2}. Its cost is 4.

Data Structures for Prim

3. Priority queue (e.g. binary heap) for the edges in the best-edge array (orange edges, from isolated nodes to the tree). Should support extract-min, decrease-key

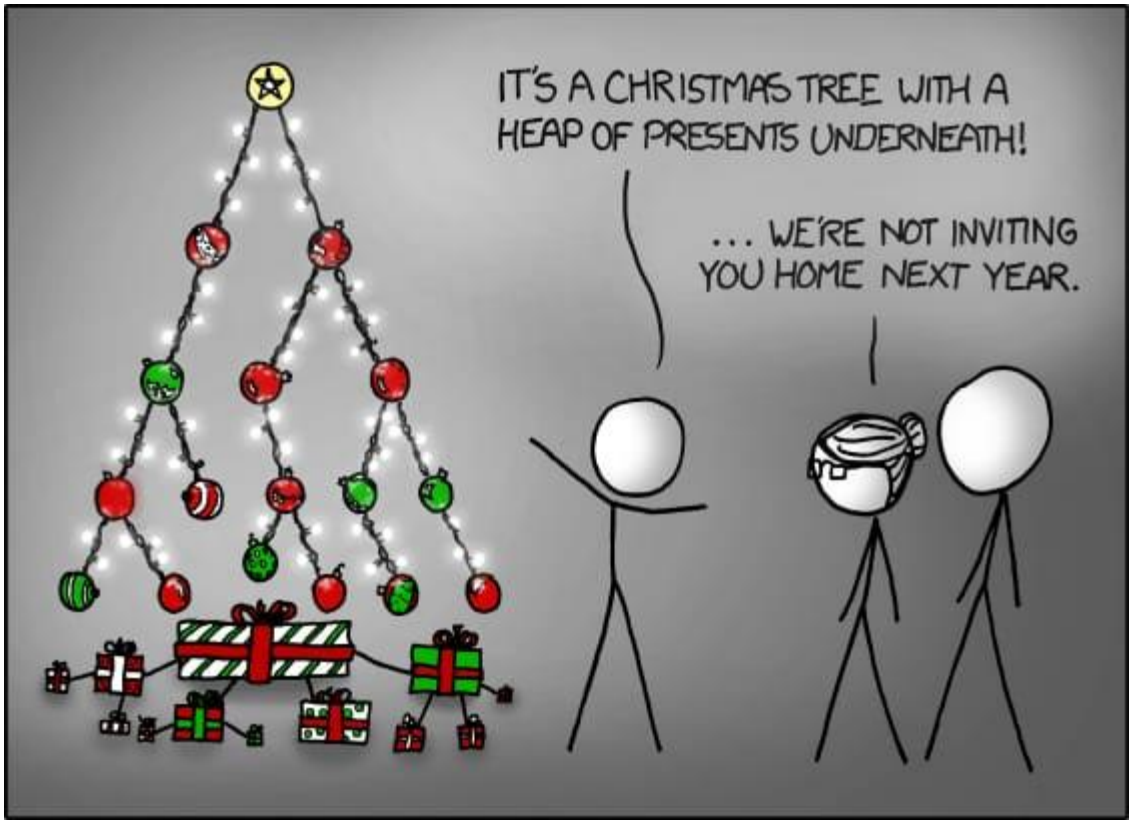




	1	2	3	4	5	6	7
parent			2	-	1	1	2
cost			3	∞	4	3	3

	1	2	3	4	5	6	7
parent			2	7	7	1	
cost			3	0	1	3	

Evaluation of Prim



Evaluation of Prim

- Priority queue: O(log n) per operation, using a binary heap.
- O(m+n) priority queue operations:
  - An edge is considered when a vertex incident to it joins the tree.
  - Every node is added to the tree once.
- Time complexity is O((m+n) log n)
  [= O(m log n) we assume G is connected]
- Can be reduced to O(m + n log n) when using Fibonacci heaps

Prim's Algorithm

1. for each u ∈ V:
2. u.cost ← ∞
3. u.parent ← null
4. r.cost ← 0
5. Q ← V
6. while Q is not empty:
7. u ← extract-min(Q)
8. for each v adjacent to u:
9. if v in Q and c(u,v) < v. cost:
10. v.parent ← u
11. v. cost ← c(u,v)
12. T = {(v, v.parent) | v ∈ V\{r} }

Initialization
r is the (arbitrary selected) first node.

Q is a min priority queue

iterations

Kruskal vs Prim

demo1
demo2

- Kruskal
  - Simple
  - Good with sparse graphs - O(m log m)
- Prim
  - More complicated
  - Better for dense graphs - O(m log n)

Note: O(log n) = O(log m)

($n-1 \leq m < n^2$ implies $\log m < \log n^2 = 2 \log n$)

Sample Problem 1

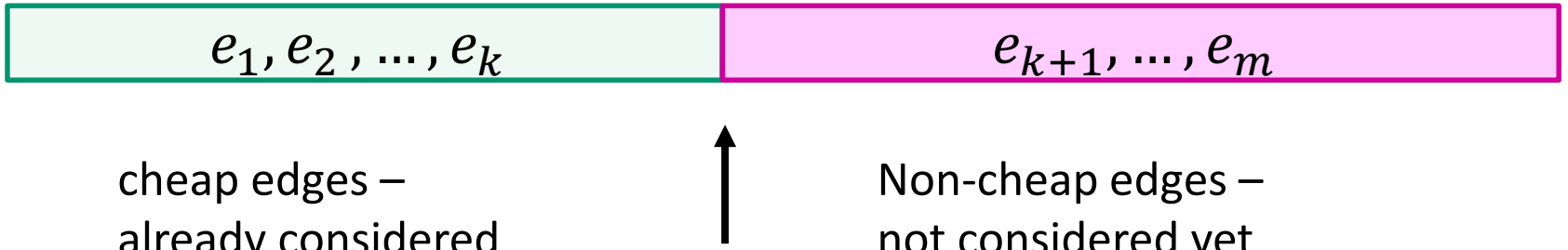
Let G be a connected undirected graph with positive edge weights. An edge is considered *cheap* if its weight is less than 10.
It is known that for two vertices $a,b \in V$, there exists a path connecting a and b consisting of cheap edges.

Prove or give a counter example: In every MST of G, the (only) path connecting a and b consists of cheap edges.

Sample Problem 1

Proof: Let T be an MST of G. Consider the run of Kruskal that produces T (in the recitation, you will prove that such a run exists for any MST T).
Consider the time-point after all cheap edges were considered. We show that a and b are already connected at this time-point – implying that the path connecting them is as required.

$$c(e_1) \leq c(e_2) \leq \cdots \leq c(e_m)$$



Sample Problem 1

Assume by contradiction that a and b are not in the same connected component. Let p be a path connecting a and b consisting of cheap edges. Since a and b are not in the same component, there must be an edge in p whose endpoints are in different components. This edge was considered already (weight < 10) and was supposed to be included in the MST – a contradiction to Kruskal's operation.



Sample problem 2

Given is an undirected connected graph $G=(V,E)$, with positive edge weights. Let α be the maximal cost of an edge, that is, $\forall e \in E$, $w(e) \leq \alpha$ and for at least one edge $w(e)=\alpha$.

Is it true that every MST of G includes the same number of edges of weight α?

If **YES**, describe an algorithm, as efficient as you can, that returns the number of edges of weight α in every MST of the graph.

If **NO**, give a counter example.

Sample problem 2

Yes.

Algorithm:

1. Remove from the graph edges having weight α.
2. Use DFS or BFS to count the number, x, of connected components in the resulting graph.
3. Return x-1

Details of step 2: Loop through the vertices, starting a new BFS or DFS whenever the loop reaches a vertex that has not been included in a previously found connected component (not marked by BFS or DFS). Increase x by 1 with every new BFS/DFS (new component).

Sample problem 2

Correctness: Assume that Kruskal's algorithm is performed on the graph. Consider the time after all edges having weight less than α were considered.
The number of Union/Find sets at this time point equals x, because any two vertices that are connected by edges having weight less than α are in the same component (by sample problem 1).

Note also that these components are the same for any run of Kruskal, and therefore for any MST. In the MST itself (after Kruskal terminates), there is a single component, therefore, x-1 edges of weight α should be added in order to merge the x components into a single one.

Sample problem 2

Complexity:

Assume the graph is given by adjacency lists:

Step 1: Scan the adj. lists to produce the graph of edges with cost less than α: O(|E|)

Step 2: DFS or BFS + marking : O(|E|)

Step 3: O(1).

All together: O(|E|).

Sample problem 3

Let G=(V,E) be a connected undirected graph with positive edge costs, $c: E \to \mathbb{R}^+$. Each of the edges is owned by an independent agent who offers the edge for sale.
For a given T, an MST of G, each of the agents who owns an edge in T is paid the cost of the edge.

Let $e \in T$ be an edge in the MST. Suggest the agent that owns e, an efficient algorithm that returns whether and by how much she can increase the cost of e, such that even at the new cost, e will continue to be part of some MST of G. If there is no limit on the increase the algorithm should announce it.

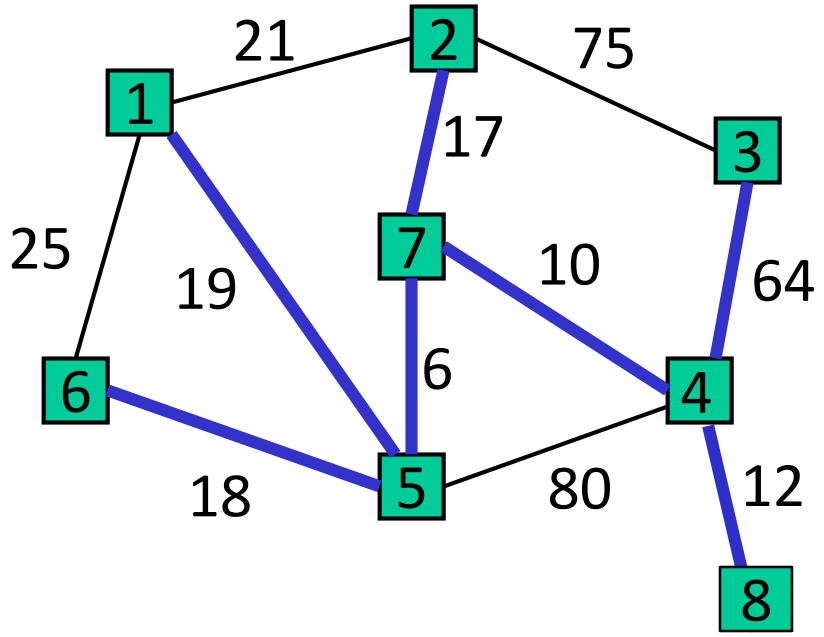
Describe the algorithm, analyze its time complexity and justify shortly.

Sample problem 3

Note: the algorithm should return the maximal $\Delta \geq 0$, such that if the cost of e is $c(e)+\Delta$ then there exists an MST that includes e.

Example:

- For e=(5,7), currently c(e)=6, if its cost increases to 21 it still belongs to some spanning tree. If its cost is more than 21 then the MST will include (1,2) instead of (5,7). Therefore, Δ=21-6=15.
- For e=(4,8), Δ is not bounded.



Sample problem 3

Algorithm:

1. Remove e from T. Let T_1,T_2 be the resulting connected components.
2. Let H be the set of edges in the cut induced by T_1 (edges that have one endpoint in T_1 and one endpoint in T_2).
3. If H={e} then $\Delta = \infty$ (unlimited increase). In other words, e is included in every MST independent of its cost.
Otherwise, let e' be the cheapest edge in H-{e}.
Return Δ=c(e')-c(e).

Sample problem 3

Time complexity:
Identify the cut edges: O(|V|+|E|)=O(|E|) [BFS or DFS, then scan the graph's data structure].
Find e': O(|H|)= O(|E|).
Compute Δ : O(1)
All together: O(|E|)

Correctness: By the blue rule if an edge is cheapest in a cut, then there exists an MST that includes it. The algorithm finds the maximal cost of e that will keep it cheapest in the cut induced by T_1. We cannot increase its cost by more than x, since then (by an exchange argument), there is a cheaper way to connect T_1 and T_2.

Sample Problem 4
(Back to Greedy Algorithms)
The doughnut-distribution problem

- n kids arrive at your Hanukkah party. You have m doughnuts. The j-th doughnut weights w_j.
- Every kid is associated with a greediness parameter g_i, denoting the minimum weight of a doughnut that makes him/her satisfied.
- Suggest an algorithm for distribution of doughnuts to kids in a way that every kids receives at most one doughnut and the number of satisfied kids is maximized.



Doughnut distribution problem



Algorithm:

- Sort the doughnuts: $w_1 \geq w_2 \geq \ldots . w_m$.
- Sort the kids: $g_1 \geq g_2 \geq \ldots . g_n$.
- Index the kids and the doughnuts according to the sorted order. A solution is given by a set of pairs (B_i, D_j) – the i-th kid gets the j-th doughnut.
- If $w_1 \geq g_1$ then pair (B_1, D_1), and continue with $w_2, \ldots, w_m$, $g_2, \ldots, g_n$.
- If $w_1 < g_1$ then don't give kid 1 any doughnut and continue with $w_1, \ldots, w_m$, $g_2, \ldots, g_n$.

Doughnut distribution problem

Greedy choice property:

Claim 1: If $w_1 \geq g_1$ then there exists an optimal solution that includes the pair (B_1, D_1) [in other words - in which the greediest kid gets the largest doughnut].



Doughnut distribution problem

Proof of Claim 1: Let A be any solution. We show that it is possible to modify A to include (B_1,D_1) without hurting the number of satisfied kids.

1. If B_1 is not happy in A, then by giving her D_1 we make her happy. Maybe we hurt someone else, but we did not hurt the total number of satisfied kids.

2. If B_1 is happy in A, assume $(B_1, D_i) \in A$. If D_1 is not given to any kid then give it to B_1, who clearly remains happy. If $(B_j, D_1) \in A$, change A by switching to the pairs $(B_1,D_1),(B_j,D_i)$. Since $w_1 \geq w_i \geq g_1 \geq g_j$, both kids remain happy after the switch.

Doughnut distribution problem

Greedy choice property:

Claim 2: if $w_1 < g_1$ then there is an optimal solution in which kid 1 is not getting a doughnut.

Proof: There is no solution in which B_1 is satisfied. A pair (B_1, D_j) can be removed from a solution without hurting the number of satisfied kids.

Using the two claims we will show (by induction on k): There exists an optimal solution in which the first k kids are given (or not given) doughnuts as in the greedy assignment.



Doughnut distribution problem



Base: k = 0. Vacuous truth.

Step: Assume there is an optimal solution that agrees with greedy on the first k-1 kids. After the distribution of doughnuts to the first k-1 kids, the optimal solution is left with the same set of doughnuts that remain available to the algorithm, say $D_q,\ldots,D_m$.

An optimal solution consists of the optimal prefix assigned so far + an optimal solution to the remaining kids with the remaining doughnuts.

The greedy choice property (claims 1 and 2) refers to B_k and D_q, as required in the induction step.

Doughnut distribution problem



In particular: After n children, there is an optimal solution where all the kids were given doughnuts as in the greedy algorithm.